

TEMA 1 – Arrays, Funciones y Objetos del Usuario

Objetivo del tema

En este tema aprenderemos sobre los Arrays, las funciones y los Objetos definidos por el usuario

1.1 Funciones predefinidas del lenguaje

Consulta la presentación **UD04 - Arrays Funciones Objetos del Usuario**.

En JavaScript, las **funciones predefinidas** (o funciones globales de nivel superior) son métodos integrados directamente en el objeto global (window en el navegador o global en Node.js). Se pueden invocar directamente desde cualquier punto del código sin necesidad de instanciar ningún objeto.

Las funcionalidades más relevantes son:

- **Conversión de cadenas a números:** Usando **parseInt(cadena, base)**. Hay que tener en cuenta que los caracteres ‘extraños’ devuelve NaN (Not a Number). Existen varios detalles más que puedes consultar en la presentación.

De la misma forma que tenemos parseInt, también tenemos otros, como **parseFloat**.

```
console.log(parseInt("100px", 10)); // 100 (Ignora el texto 'px')
console.log(parseInt("3.14", 10)); // 3 (Trunca los decimales, no redondea)
console.log(parseInt("Texto 100", 10)); // NaN (Empieza por texto)
```

- **Inspección de valores:** Al usar **isNaN(valor)** se evalúa si un valor evaluado es NaN.

```
console.log(isNaN("Hola")); // true (Intenta convertirlo a número, falla y da NaN)
console.log(isNaN("123")); // false (Lo convierte a 123, Sí es número)
console.log(isNaN(NaN)); // true
```

- **Evaluar cadena:** Es posible comprobar una cadena de texto (*string*) y **ejecutarla** dinámicamente como si fuera código fuente de JavaScript. Obviamente, es desaconsejable utilizarla en tus proyectos.

```
const x = 10;
const y = 20;

const resultado = eval("x + y");
console.log(resultado); // Salida: 30
```

1.2 Funciones definidas por el usuario

El usuario (programador) también puede definir sus propias funciones. La estructura básica de una función es la siguiente:

```
function nombre([argumentos]) {  
    //Sentencias  
    //[return dato]  
}
```

Donde:

- **Function:** palabra reservada para definir una función.
- **Nombre:** palabra que determina el nombre de la función.
- **Argumentos** (opcional): parámetros que pueden utilizarse internamente en la función.
- **Return:** la función puede o no devolver un valor, que estaría indicado después de la palabra reservada return.

Arrow Functions

Las **Arrow Functions** (funciones flecha), son una sintaxis alternativa y compacta para escribir expresiones de función en JavaScript. Esto **NO** significa que son un truco para escribir menos. Tienen diferencias de comportamiento respecto a las funciones tradicionales. Para empezar, no se pueden usar como constructores.

Ejemplo:

```
const sumar = (a, b) => {  
    return a + b;  
};
```

La misma función, de forma resumida:

```
const sumar = (a, b) => a + b;
```

Estas funciones equivalen ambas a poner:

```
var sumar = function () {  
    return a + b;  
}
```

La utilidad más elemental de este tipo de funciones es hacer cosas como crear objetos de forma anónima de una sola vez, y operaciones simples de transformación de tipos de datos.

```
const crearPersona = (nombre, edad) => ({ nombre: nombre, edad: edad });
```

También se usan mucho como manejadores de eventos.

Infinitos parámetros

Es posible pasar un número indeterminado de parámetros a una función. Para ello usamos el operador (...) al último parámetro de la función. Esto le indica a JavaScript que agrupe todos los argumentos "restantes" pasados a la función dentro de un **Array**.

Ejemplo:

```
function sumatorio (...sumandos) {  
    let total = 0;  
    for (const arg of sumandos) {  
        total += arg;  
    }  
    return total;  
}
```

Independientemente del número de parámetros que le pasamos a sumatorio, para la función todos ellos son un Array llamado sumandos. Esto nos permite recorrerlos con un bucle sencillo.

1.3 Arrays

Un **Array** en JavaScript es un **objeto** global especial y reutilizable que se utiliza para almacenar colecciones ordenadas de datos. A diferencia de Java o C, un Array en JavaScript **NO** es un bloque contiguo de memoria fija ni de tipo único; es una estructura dinámica, de tamaño variable y heterogénea.

Por tanto, podemos almacenar datos de **diferentes** tipos en un mismo array.

Tienes un listado completo de sus atributos y métodos en la presentación **UD04 - Arrays Funciones Objetos del Usuario**. Esto permite usar al Array como una estructura de acceso aleatorio, como una Pila (LIFO) o una Cola (FIFO).

1.4 Objetos definidos por el usuario

Es posible definir los objetos como nosotros queramos. Ya hablamos de esto en el siguiente documento:

0612 - Desarrollo Web en Entorno Cliente - UD3.3 - Objetos en JavaScript

Ejercicio 1 – Funciones

Crea las siguientes funcionalidades:

- Crea una *función anónima* que calcule el cuadrado de un número y la almacene en una variable.
- Luego, ejecuta esa función con el número 5 y muestra el resultado en consola.
- Convierte la función en una función *auto invocada*.

Ejercicio 2 – Arrow Functions

Crea las siguientes funcionalidades:

- Transforma la función del ejercicio 1 en una Arrow Function.
- Ejecuta la función con un número diferente (por ejemplo, 8) y muestra el resultado en consola.
- Si solo tienes un parámetro, ¿puedes eliminar algo en la declaración de la función?

Ejercicio 3 – Parámetros por defecto

Crea las siguientes funcionalidades:

- Crea una función que acepte dos parámetros: un número y un porcentaje (por defecto será 10%).
- La función debe devolver el valor resultante de aplicar el porcentaje al número.
- Prueba la función con y sin pasar el segundo parámetro.

Ejercicio 4 – REST

Crea las siguientes funcionalidades:

- Crea una función que acepte un número indefinido de parámetros usando el operador REST (...).
- La función debe devolver la suma de todos los parámetros.
- Prueba la función con diferentes cantidades de números.

Ejercicio 5 – Combina todo

Crea las siguientes funcionalidades:

- Crea una *Arrow Function* que acepte una lista de números y un parámetro opcional (valor por defecto: 1). La función debe multiplicar cada número por el parámetro opcional.
- Usa el operador *REST* para aceptar cualquier cantidad de números como argumento.
- Muestra el resultado de la multiplicación de todos los números en consola.

Ejercicio 6 – Arrays

Vamos a gestionar una lista de países haciendo uso de Arrays. Para ello necesitarás crear un archivo `arrays.js` que incluya las siguientes funciones:

1. Mostrar el número de elementos del array.
2. Mostrar todos los elementos del array.
3. Muestra los elementos del array en sentido inverso.
4. Muestra los elementos del array ordenados alfabéticamente (pero no los ordena).
5. Añadir un elemento al principio del array.
6. Añadir un elemento al final del array.
7. Borrar un elemento al principio del array (y decir cuál se ha borrado).
8. Borrar un elemento al final del array (y decir cuál se ha borrado).
9. Muestra el elemento que se encuentra en una posición que el usuario indica.
10. Muestra la posición en la que se encuentra un elemento que le indica el usuario.
11. Muestra los elementos que se encuentran en un intervalo que el usuario indica.

Ten en cuenta que el array será una variable global y que se pasará por parámetro en todas las funciones.

Cuando el usuario cargue la página, se cargará un prompt donde se mostrarán (en el prompt, no en la página) las opciones:

1. Mostrar número de países.
2. Mostrar listado de países (y le preguntará si quiere mostrarlos en el orden que se encuentran en el array, del revés u ordenados alfabéticamente).
3. Mostrar un intervalo de países (y le pedirá que introduzca el intervalo en formato inicio-fin; luego deberás extraer el valor inicio y fin).
4. Añadir un país (y le preguntará si quiere añadir al principio o al final).
5. Borrar un país (y le preguntará si quiere borrar al principio o al final).
6. Consultar un país (y le preguntará si quiere consultar por posición o por nombre).

Todas las operaciones que se realicen se irán mostrando en alerts.

Ejercicio 7 – Objetos

Necesitamos almacenar en un programa todos los discos de música que tenemos en casa. Ahora que sabemos crear nuestros propios objetos es el mejor modo de guardar esta información.

1. Crea una clase “disco” que almacene la siguiente información:
 - Nombre del disco.
 - Grupo de música o cantante.
 - Año de publicación.
 - Tipo de música (podrá ser “rock”, “pop”, “punk” o “indie”);
 - Localización: almacenará un número de estantería.
 - Prestado: almacenará un valor booleano. Por defecto será false.

Además, tendrá los siguientes métodos:

- Un constructor sin parámetros (las 4 primeras propiedades serán cadenas vacías, la localización será 0 por defecto y prestado estará a false).
- Getters y setters de todas las propiedades salvo de Prestado.
- Un método que permita incluir las cinco primeras propiedades a la vez; la propiedad prestado seguirá a false.
- Un método que permitirá cambiar la propiedad Prestado, si está en false a true y viceversa.
- Un método que muestre toda la información de un disco.

Guarda todo el código en un archivo llamado **disco.js**

2. Carga en tu página el archivo arrays.js que hicimos en la práctica anterior.
3. Crea un array vacío para almacenar los discos.
4. Cuando el usuario cargue la página, se cargarán las opciones:
 - Mostrar número de discos.
 - Mostrar listado de discos (y le preguntará si quiere mostrarlos en el orden que se encuentran en el array, del revés u ordenados alfabéticamente).
 - Mostrar un intervalo de discos (y le pedirá que introduzca el intervalo en formato inicio-fin; luego deberás extraer el valor inicio y fin).
 - Añadir un disco (y le preguntará si quiere añadir al principio o al final).
 - Borrar un disco (y le preguntará si quiere borrar al principio o al final).
 - Consultar un disco (y le preguntará si quiere consultar por posición o por nombre).

Todas las operaciones que se realicen se irán mostrando con alerts.